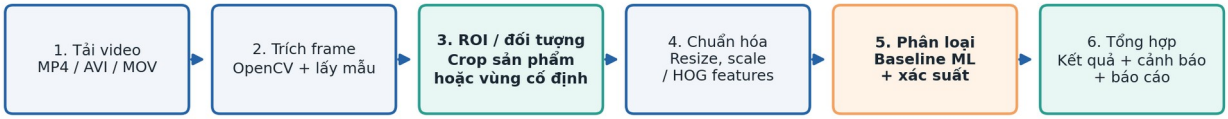
PROJECT
PROPOSAL

ConveyorVision

Hệ thống phân loại sản phẩm trên băng chuyền ứng dụng học máy có giám sát và thị giác máy tính

PIPELINE ĐỀ XUẤT — từ video đến kết quả kiểm tra



Lưu ý: frame không đồng nghĩa với một sản phẩm. Muốn đếm sản phẩm thật cần ROI/track ID hoặc quy tắc đếm riêng.

HẠNG MỤC	ĐỀ XUẤT
Loại sản phẩm	Prototype web: tải video, phân tích khung hình/ROI, hiển thị kết quả
Baseline	Majority Class → Logistic Regression → HOG + Linear/RBF SVM → Random Forest trên đặc trưng
Mục tiêu đánh giá	Ưu tiên Recall lớp “sản phẩm lỗi”, đồng thời theo dõi Precision, F1, FN và thời gian xử lý
Trạng thái	Proposal kỹ thuật — các chỉ số hiệu năng sẽ được đo sau khi có dữ liệu thật

Bản proposal và khung baseline để thảo luận / triển khai MVP

1. Tóm tắt đề xuất

Executive summary

ConveyorVision là prototype web hỗ trợ kiểm tra sản phẩm trên băng chuyền từ video. Hệ thống nhận video do người dùng tải lên, trích xuất frame bằng OpenCV, xác định vùng cần phân tích (ROI), tiền xử lý, chạy mô hình phân loại đạt/lỗi và trả kết quả cùng các chỉ số đánh giá.

Điểm cần chốt sớm: Một frame không luôn tương đương một sản phẩm. Nếu một sản phẩm xuất hiện ở nhiều frame, cộng số frame dự đoán lỗi sẽ đếm lặp. MVP phải chọn rõ một trong hai chế độ: (A) phân loại ROI cố định / mỗi ảnh chứa một sản phẩm; hoặc (B) detect/track từng sản phẩm rồi tổng hợp theo ID.

Mục tiêu

- Xây dựng pipeline dữ liệu từ video đến frame/ROI có nhãn và có thể tái lập.
- So sánh baseline đơn giản, giải thích được trước khi xem xét deep learning.
- Tối ưu ngưỡng quyết định trên validation để giảm bỏ sót sản phẩm lỗi (FN).
- Cung cấp web demo có luồng upload, trạng thái xử lý, kết quả và báo cáo.

Phạm vi MVP

Trong phạm vi	Ngoài phạm vi ở phiên bản đầu
Video file upload; MP4/AVI/MOV theo cấu hình hỗ trợ	Tích hợp camera công nghiệp real-time / PLC / cơ cấu gạt tự động
Nhị phân: 0 = đạt, 1 = lỗi; hỗ trợ xem xác suất/score	Phân loại nhiều loại lỗi chi tiết khi chưa có nhãn đủ tin cậy
OpenCV sampling + ROI rõ ràng; baseline scikit-learn	Cam kết hiệu năng trên mọi loại băng chuyền/ánh sáng
Metric, confusion matrix, xuất JSON/CSV	Triển khai production có SLA/cybersecurity audit đầy đủ

Tên sử dụng

Tên sản phẩm: ConveyorVision. Tên học thuật đề xuất: “Hệ thống phân loại sản phẩm trên băng chuyền ứng dụng học máy có giám sát”.

2. Bài toán, giả định và tiêu chí thành công

Phát biểu bài toán

Với mỗi đơn vị phân tích ảnh x (một ảnh sản phẩm hoặc một ROI đã xác định), hệ thống dự đoán nhãn $y \in \{0: \text{đạt}, 1: \text{lỗi}\}$ và score s biểu thị mức độ nghi ngờ thuộc lớp lỗi. Với threshold t, kết quả lỗi nếu $s \geq t$; ngược lại là đạt. Với SVM không có xác suất trực tiếp, dùng decision score hoặc hiệu chỉnh xác suất riêng nếu cần diễn giải như xác suất.

Giả định vận hành cần xác nhận

- Camera tương đối cố định; kích thước và vị trí sản phẩm có độ biến thiên có thể quản lý.
- Có nhãn do người kiểm tra xác nhận; định nghĩa "lỗi" được viết thành hướng dẫn gán nhãn.
- Video có quyền sử dụng cho mục đích học tập / demo; không đưa dữ liệu nhạy cảm không cần thiết vào hệ thống.
- Nếu mục tiêu là đếm sản phẩm vật lý, phải có ROI/track ID hoặc quy tắc lấy một mẫu cho mỗi sản phẩm.

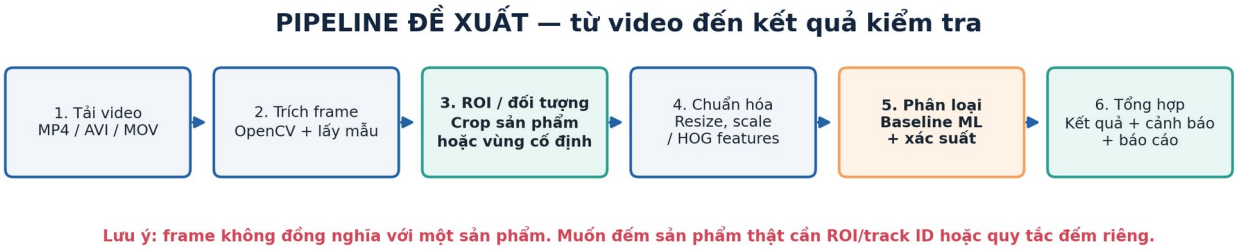
Tiêu chí nghiệm thu đề xuất (định lượng sau pilot)

Nhóm	Tiêu chí	Cách xác nhận
Dữ liệu	Có manifest, nhãn, ID video/phiên quay và quy tắc chia tập	Kiểm tra schema và audit mẫu nhãn
ML baseline	So sánh majority baseline với ít nhất 3 mô hình ML	Bảng metric trên cùng split / cross-validation
Lỗi bỏ sót	Báo cáo Recall lớp lỗi và số FN; chọn threshold bằng validation	Confusion matrix + PR curve / threshold sweep
Ứng dụng	Upload → xử lý → trang kết quả hoạt động trên demo	Test luồng thành công và file không hợp lệ
Tính trung thực	Không dùng test set để chọn mô hình/ngưỡng	Lưu cấu hình, seed, phiên bản dữ liệu và model

Không đặt trước điểm số giả: Chưa có video/nhãn thực tế nên chưa thể báo cáo Recall, Precision, tốc độ hoặc độ chính xác kỳ vọng như kết quả đo. Các con số hiệu năng chỉ được điền sau khi chạy thử nghiệm.

3. Pipeline tổng thể

Quy trình xử lý từ file video đến kết quả web



Hình 1. Luồng suy luận đề xuất; khâu ROI/dedup phụ thuộc cách sản phẩm xuất hiện trong video.

Mô tả luồng

Bước	Đầu vào → đầu ra	Điểm kiểm soát
1. Upload	Video → file tạm + metadata	Kiểm tra đuôi/MIME, dung lượng, codec; đặt tên file an toàn
2. Sampling	Video → frame theo bước thời gian / FPS	Chọn sampling rate đủ bắt sản phẩm nhưng tránh frame trùng gần nhau
3. ROI	Frame → crop/ảnh sản phẩm	ROI cố định cho demo đơn giản; tracking/detection nếu cần đếm sản phẩm
4. Preprocess	ROI → tensor/vector đặc trưng	Resize đồng nhất; RGB/gray; scale pixel [0,1] hoặc scaler fit trên Train
5. Predict	Vector → nhãn + score	Dùng đúng pipeline/model đã lưu; threshold lấy từ validation
6. Aggregate	Dự đoán → summary / file kết quả	Chỉ đếm theo sản phẩm/ID nếu có cơ chế chống đếm lặp

Hai mức MVP có thể chọn

Mức	Cách làm	Ưu / hạn chế
MVP-A: ROI cố định	Mỗi frame đã căn sản phẩm trong vùng cố định hoặc mỗi ảnh tương ứng một sản phẩm	Nhanh, ít thành phần; chỉ phù hợp camera và bối cảnh ổn định
MVP-B: đối tượng theo ID	Phát hiện / tracking sản phẩm trước, phân loại ROI theo ID	Có cơ sở đếm theo sản phẩm; phức tạp hơn và cần kiểm tra chất lượng tracking

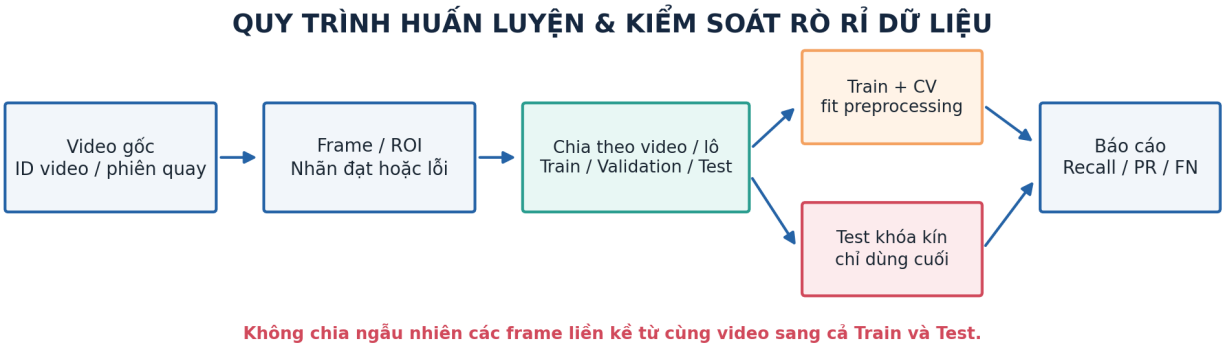
4. Dữ liệu, gán nhãn và chống rò rỉ

Schema dữ liệu tối thiểu

Trường	Ví dụ / mô tả	Bắt buộc?
sample_id	Mã duy nhất của ảnh hoặc ROI	Có
video_id / lot_id	ID video, phiên quay hoặc lô sản xuất	Có, nếu có nguồn video
frame_idx / timestamp	Chỉ số frame hoặc thời gian trích xuất	Có
roi_path	Đường dẫn ảnh gốc/crop đã lưu	Có
label	0 = đạt; 1 = lỗi	Có
defect_type	Loại lỗi cụ thể nếu có nhãn đáng tin cậy	Tùy chọn
annotator / review_status	Người gán nhãn, trạng thái kiểm tra	Khuyến nghị

Quy tắc gán nhãn

- Viết checklist lỗi kèm ảnh ví dụ; thống nhất xử lý trường hợp mờ, che khuất hoặc không thể kết luận.
- Ưu tiên review kép một phần dữ liệu và ghi lại bất đồng; không tự động coi ảnh mơ hồ là “đạt”.
- Trích frame có kiểm soát để tránh hàng trăm ảnh gần giống nhau làm phồng kích thước tập dữ liệu.
- Giữ nguyên quan hệ giữa ảnh và video/lot gốc để split theo nguồn quay.



Hình 2. Chia tập theo nhóm nguồn quay để giảm rò rỉ do các frame liền kề gần như giống nhau.

Chia dữ liệu khuyến nghị: Tách Train / Validation / Test theo video, phiên quay hoặc lô sản xuất thay vì chia ngẫu nhiên từng frame. Nếu đủ số nhóm, dùng StratifiedGroupKFold để cố gắng giữ tỷ lệ lớp đồng thời không tách cùng nhóm sang các fold.

5. Thiết kế baseline học máy

Baseline nên tăng dần độ phức tạp, để biết phần cải thiện đến từ đặc trưng ảnh, mô hình hay pipeline. Tất cả mô hình dùng cùng tập dữ liệu/split và cùng định nghĩa positive = “lỗi”.

ID	Baseline	Biểu diễn dữ liệu	Điểm mạnh / lưu ý
B0	Majority Class	Luôn dự đoán lớp phổ biến nhất	Mốc kiểm tra tối thiểu; có thể Accuracy cao nhưng Recall lớp lỗi = 0
B1	Logistic Regression	Resize ảnh xám/RGB → flatten; scale pixel và StandardScaler trong Pipeline	Nhanh, có score; số chiều cao, nhạy với cách chuẩn hóa và ánh sáng
B2	HOG + Linear SVM	HOG mô tả gradient/biên → StandardScaler → LinearSVC	Baseline ảnh cổ điển dễ tái lập, thường phù hợp khi dữ liệu nhỏ/vừa
B3	HOG + RBF SVM	HOG → StandardScaler → SVC(RBF)	Biên phi tuyến; cần dò C/gamma và cân bằng lớp
B4	Random Forest	HOG và/hoặc đặc trưng màu/thống kê	Không bắt buộc scale; dễ kiểm tra tầm quan trọng tương đối, nhưng không mặc định tốt hơn SVM

Pipeline huấn luyện

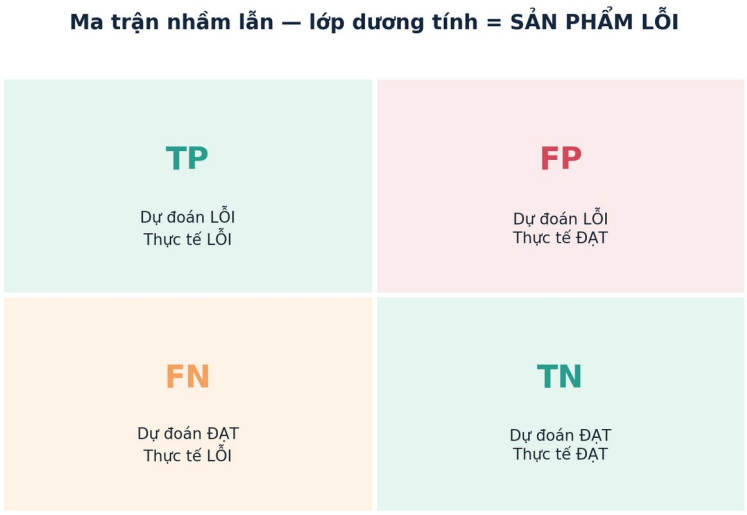
- Bất kỳ scaler, feature selector hoặc bước học đặc trưng nào có tham số phải fit chỉ trên Train/fold huấn luyện.
- Dùng Pipeline/ColumnTransformer của scikit-learn để preprocessing được lặp lại giống hệt lúc inference.
- Dùng class_weight="balanced" cho mô hình hỗ trợ nếu dữ liệu lệch lớp; chỉ giữ khi validation cho thấy có ích.
- Dò hyperparameter bằng cross-validation trên Train; dùng nhóm video/lot nếu nhiều frame từ cùng nguồn.
- Lưu pipeline đã fit, threshold đã chọn, config, phiên bản dữ liệu và nhãn lớp.

Cấu hình khởi động (để chạy pilot)

Tham số	Khởi điểm gợi ý	Cách tinh chỉnh
Resize	Ảnh xám 128×128 cho B1; thử kích thước khác nếu chi tiết nhỏ	Chọn trên validation; không xem test để tinh chỉnh
HOG	orientations=9; pixels_per_cell=(8,8); cells_per_block=(2,2)	Dò ít cấu hình, ghi rõ kết quả
SVM	LinearSVC(C=1) và SVC(RBF, class_weight="balanced")	Grid nhỏ cho C / gamma
Random Forest	n_estimators=300; class_weight="balanced"	Dò max_depth / min_samples_leaf

Lưu ý về scaling: Pixel có thể chia 255 để đưa về [0,1]. StandardScaler không bắt buộc với mọi mô hình; nó phù hợp với LR/SVM và cần fit trong fold Train. Cây quyết định/Rừng ngẫu nhiên thường không cần chuẩn hóa.

6. Đánh giá mô hình và chọn threshold



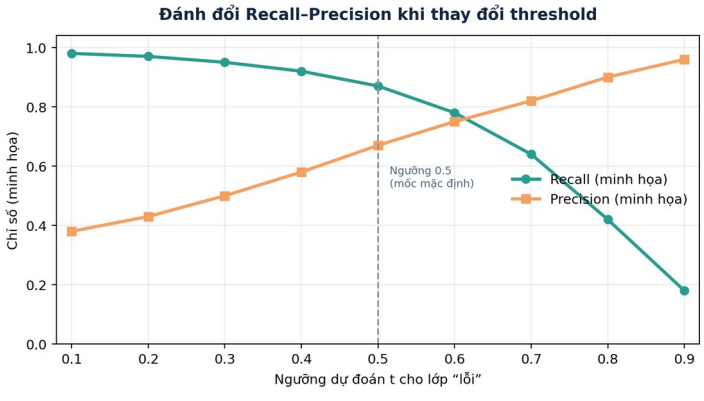
Hình 3. Confusion matrix; lớp dương tính được định nghĩa là sản phẩm lỗi.

Chỉ số cần báo cáo

Chỉ số	Công thức / ý nghĩa	Vai trò
Recall lỗi	TP / (TP + FN)	Ưu tiên cao: trong các sản phẩm lỗi thật, phát hiện được bao nhiêu
Precision lỗi	TP / (TP + FP)	Trong các dự đoán lỗi, bao nhiêu là lỗi thật; kiểm soát báo động nhầm
F1	Trung bình điều hòa Precision và Recall	Dùng so sánh tổng hợp; không thay thế ưu tiên nghiệp vụ
Specificity	TN / (TN + FP)	Khả năng nhận đúng hàng đạt
PR-AUC / Average Precision	Diện tích / tổng hợp đường Precision–Recall	Có ích khi lớp lỗi hiếm; so sánh score độc lập với một threshold
FN, FP tuyệt đối	Số trường hợp bỏ sót / báo nhầm	Giúp quy đổi thành khối lượng cần kiểm tra lại
Latency	Thời gian xử lý video hoặc ROI	Đo riêng trên môi trường thử nghiệm đã ghi cấu hình

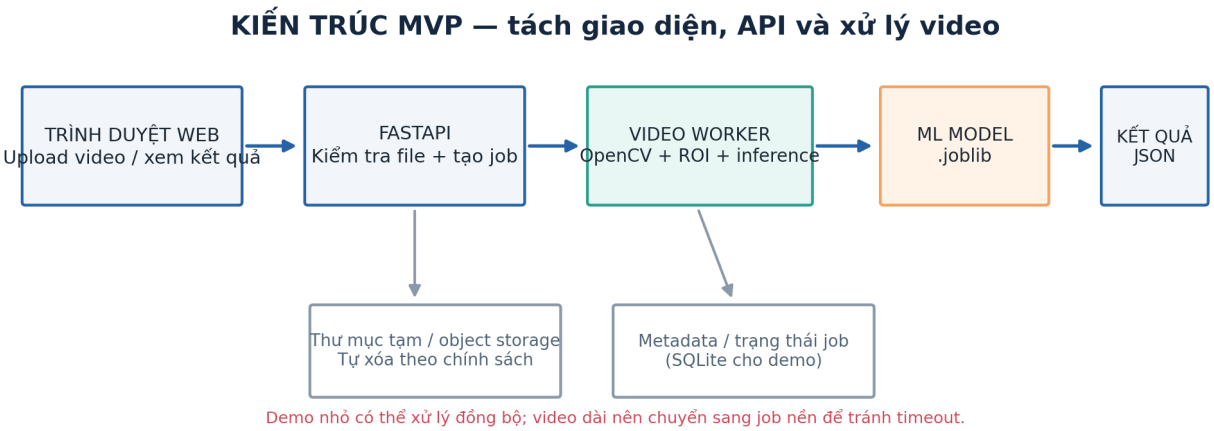
Quy trình chọn threshold

1. Huấn luyện trên Train; lấy score trên Validation hoặc dự đoán out-of-fold.
2. Quét threshold t; ghi Recall, Precision, FN và FP. Không mặc định t = 0.5 là tối ưu.
3. Chọn t theo yêu cầu nghiệp vụ về Recall và chi phí FP/FN đã thống nhất.
4. Khóa mô hình + t; chỉ dùng Test một lần cho báo cáo cuối.



Đường cong giả lập để minh họa nguyên lý; phải thay bằng kết quả validation thực tế.

7. Kiến trúc ứng dụng và API



Hình 4. Kiến trúc triển khai MVP; có thể dùng SQLite cho metadata và thư mục tạm cho video ở demo.

Stack đề xuất

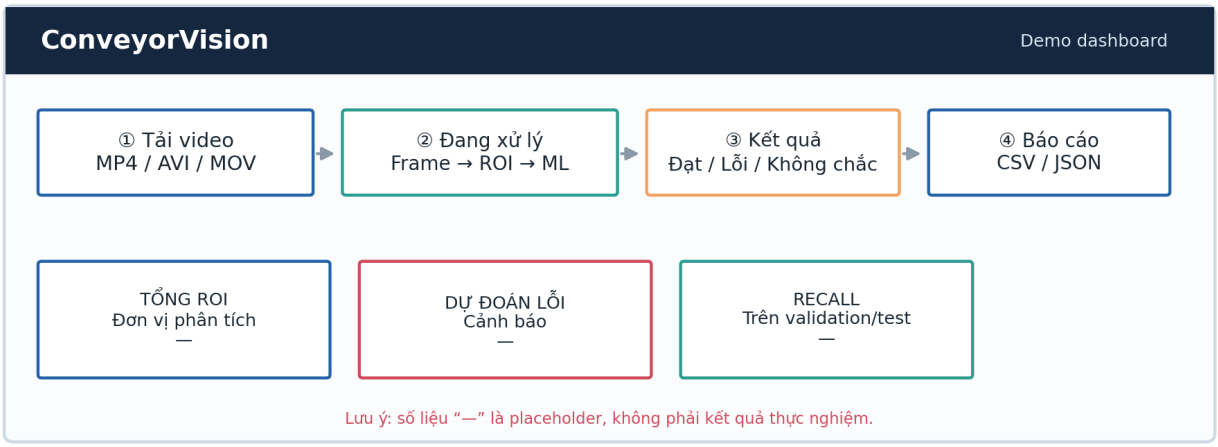
Lớp	Công nghệ	Trách nhiệm
Frontend	HTML/CSS/JS hoặc React	Upload, progress/status, xem preview và kết quả
API	FastAPI	Validate request, trả job ID, cung cấp endpoint kết quả
Video / CV	OpenCV, NumPy, scikit-image (nếu dùng HOG)	Đọc video, lấy frame, ROI, tiền xử lý và đặc trưng
ML	scikit-learn + joblib	Pipeline inference, score và threshold
Lưu trữ demo	Thư mục riêng + SQLite	File tạm, metadata, status; có chính sách xóa

API tối thiểu

Method / path	Mục đích	Response rút gọn
POST /api/videos	Upload video, tạo job	{ "job_id": "...", "status": "queued" }
GET /api/jobs/{job_id}	Tra trạng thái xử lý	{ "status": "processing|done|failed" }
GET /api/jobs/{job_id}/results	Lấy tổng hợp kết quả	summary, predictions, metrics nếu có nhãn thật
GET /api/health	Kiểm tra dịch vụ	{ "status": "ok" }

Thiết kế an toàn tối thiểu: Giới hạn dung lượng/thời lượng; whitelist định dạng; không tin tên file do client gửi; xử lý codec lỗi; đặt timeout; lưu file ngoài thư mục public; xóa file tạm theo TTL; không chạy video upload như code.

8. Giao diện demo và định dạng kết quả



Hình 5. Wireframe giao diện đề xuất; ô số liệu là placeholder, không phải hiệu năng thực nghiệm.

Nội dung cần hiển thị

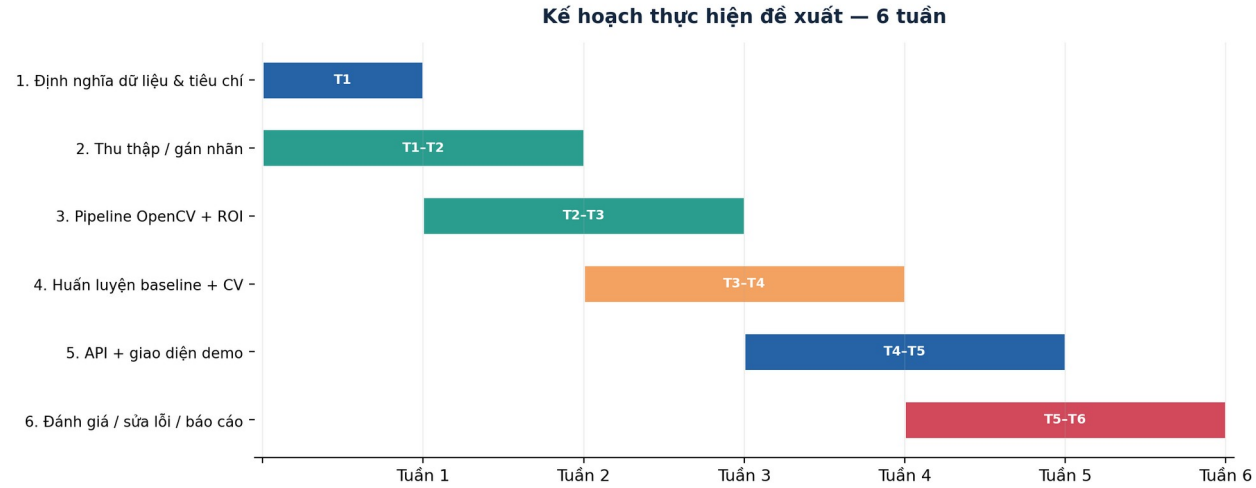
- Trạng thái job: đang chờ / đang xử lý / hoàn tất / thất bại kèm lý do dễ hiểu.
- Tóm tắt số đơn vị phân tích (ROI hoặc product ID), số dự đoán đạt, số dự đoán lỗi và số không chắc nếu dùng vùng review.
- Danh sách kết quả kèm timestamp/frame index, label, score, thumbnail và threshold đang dùng.
- Biểu đồ phân bố score, confusion matrix khi người dùng cung cấp ground truth, và chỉ số Recall/Precision khi đủ nhãn.
- Tải CSV/JSON; phân biệt rõ "số frame/ROI đã phân loại" với "số sản phẩm vật lý" nếu chưa có tracking.

Ví dụ JSON (minh họa cấu trúc)

```
{
  "job_id": "demo-001",
  "status": "done",
  "analysis_unit": "roi",
  "threshold": 0.35,
  "summary": {"processed_rois": 120, "predicted_defect": 9, "predicted_pass": 111},
  "predictions": [
    {"frame_idx": 42, "timestamp_s": 1.40, "label": "defect", "score": 0.82}
  ],
  "note": "Counts are ROI predictions, not physical product counts unless deduplicated."
}
```

9. Kế hoạch thực hiện và deliverables

Lộ trình tham khảo cho một nhóm nhỏ / đồ án môn học



Hình 6. Lịch 6 tuần đề xuất; có thể điều chỉnh theo thời gian thu thập nhãn và độ phức tạp của ROI.

Tuần	Công việc chính	Deliverable / đầu ra
1	Chốt định nghĩa lỗi, yêu cầu demo, nguồn dữ liệu, cấu trúc repo	Data card, checklist label, yêu cầu API/UI
2	Thu thập video, sampling, gán nhãn và audit chất lượng	Dataset version 0.1 + manifest + ví dụ ROI
3	OpenCV pipeline, split theo nguồn video, B0/B1/B2 baseline	Script tiền xử lý + bảng metric ban đầu
4	Dò tham số, threshold sweep, CV, chốt ứng viên	Notebook/report đánh giá + model artifact
5	FastAPI, upload/status/result, giao diện demo	Web demo end-to-end
6	Kiểm thử, đo latency, hoàn thiện báo cáo và slide	Final report, slide, README, model card

Cấu trúc repository gợi ý

```
conveyorvision/
  data/                # không commit video nhạy cảm; manifest tách riêng
  notebooks/           # EDA, baseline experiments
  src/conveyorvision/  # video_io.py, roi.py, features.py, train.py, evaluate.py
  app/                 # main.py (FastAPI), schemas.py, services.py
  models/               # pipeline.joblib + metadata.json
  tests/                # unit tests cho sampling/API/threshold
  reports/              # figures, metrics, model card
  README.md  requirements.txt  .gitignore
```

10. Rủi ro, giới hạn và cách giảm thiểu

Rủi ro	Tác động	Biện pháp giảm thiểu
Dữ liệu lệch lớp	Accuracy cao nhưng bỏ sót lớp lỗi	Recall/PR-AUC/FN, class_weight, sampling có chủ đích
Leakage giữa frame	Metric validation/test cao giả tạo	Chia theo video/lot; StratifiedGroupKFold nếu phù hợp
ROI sai / sản phẩm chồng lấp	Mô hình học nền hoặc dự đoán sai vật thể	Kiểm tra ROI overlay, audit mẫu lỗi; định nghĩa rõ chế độ MVP
Ánh sáng / camera thay đổi	Hiệu năng giảm ngoài miền dữ liệu	Thu dữ liệu đa ca, đo theo từng điều kiện; xem xét drift
Đếm lặp cùng sản phẩm	Số lỗi/tổng sản phẩm bị phóng đại	Track/deduplicate theo ID hoặc báo chính xác là số ROI/frame
Video lớn / codec lỗi	Timeout, treo worker hoặc thiếu frame	Giới hạn file, timeout, status job, logs và test nhiều codec
Nhãn không nhất quán	Recall/Precision không phản ánh chất lượng thật	Guideline, review kép một phần, theo dõi bất đồng

Giới hạn cần nêu khi bảo vệ

- Baseline cổ điển phân loại ROI, không tự động giải quyết phát hiện và tracking mọi vật thể trong cảnh.
- Kết quả chỉ có ý nghĩa đối với dữ liệu/camera/điều kiện đã kiểm tra; không tuyên bố tổng quát hóa khi chưa có test theo ca/lô độc lập.
- Score mô hình không mặc nhiên là xác suất đã hiệu chỉnh; nếu UI ghi “xác suất”, cần kiểm tra calibration.
- Hệ thống hỗ trợ quyết định kiểm tra; không điều khiển cơ cấu loại sản phẩm tự động trong MVP.

Kết quả cuối cùng dự kiến

Một web demo chạy được từ upload đến báo cáo; bộ dữ liệu đã ghi nguồn và nhãn; baseline có thể tái lập; bảng so sánh mô hình; threshold được chọn từ validation; báo cáo metric trên test khóa kín; và tài liệu README để dựng môi trường.

11. Phụ lục — khung baseline và tài liệu tham khảo

Pseudo-code pipeline đánh giá

```
1. Load manifest: X (ROI paths), y (0=pass, 1=defect), groups (video_id/lot_id)
2. Split groups -> Train / Validation / Test; khóa Test
3. For each candidate baseline:
     a. Build preprocessing + estimator as one sklearn Pipeline
     b. Run group-aware CV on Train; tune hyperparameters only here
     c. Fit on Train; get defect scores on Validation
     d. Sweep thresholds; choose t from Recall / Precision / FN trade-off
4. Select candidate + t; freeze both
5. Evaluate once on Test; report confusion matrix, Recall, Precision, F1,
   PR-AUC/AP, FN/FP and latency
6. Save fitted pipeline, class mapping, threshold, dataset/config version
```

Những quyết định chưa thể chốt nếu chưa có data

- Kích thước ảnh/ROI và cách lấy mẫu phù hợp với tốc độ băng chuyền.
- Threshold mục tiêu và mức Recall chấp nhận được cùng chi phí FP/FN.
- Sử dụng ROI cố định hay detection/tracking, tùy yêu cầu đếm từng sản phẩm.
- Số lượng mẫu cần thu thập và metric mục tiêu dựa trên pilot, độ đa dạng camera/lô và độ hiếm lỗi.

Tài liệu tham khảo kỹ thuật

[1] scikit-learn — Model evaluation: classification metrics, precision/recall, confusion matrix. https://scikit-learn.org/stable/modules/model_evaluation.html

[2] scikit-learn — Cross-validation, StratifiedGroupKFold. https://scikit-learn.org/stable/modules/cross_validation.html

[3] FastAPI — Request files / UploadFile. https://fastapi.tiangolo.com/tutorial/request-files/

[4] OpenCV — Video I/O (VideoCapture) documentation. https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html

Thông điệp khi trình bày: Điểm mạnh của đề xuất không phải hứa trước Accuracy cao, mà là một pipeline có kiểm soát dữ liệu, baseline có thể tái lập, ưu tiên Recall lớp lỗi và tách bạch rõ “frame/ROI” với “sản phẩm vật lý”.